VLANs (Virtual Local Area Networks) can be seen as a way to divide a single physical switch into multiple, isolated logical switches. This achieves three critical goals:

- **Creates smaller broadcast domains** to reduce unnecessary network traffic.
- **Enhances local security** by isolating sensitive departments from one another.
- **Allows devices attached to the same physical switch** to belong to entirely different IPv4 networks.

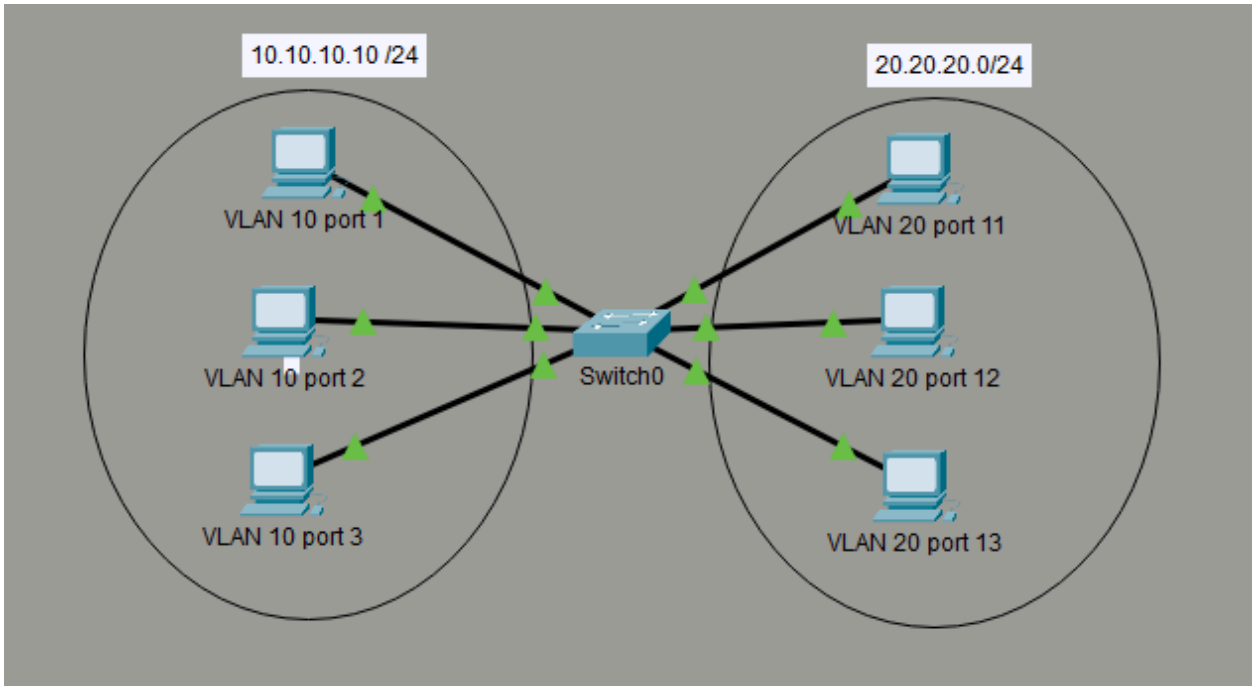
VLANs On Switches

Access ports are switch ports that are connected directly to end devices (hosts like PCs, printers, or servers).

The hosts themselves have absolutely no idea what VLAN is assigned to the switch port they are connected to. As far as the PC is concerned, it is just plugged into a standard network. The VLAN assignment (and this is critical) **lives entirely on the switch itself**.

Let's look at this in an actual network. Here, we have a single physical switch running two isolated VLANs: **VLAN 10** and **VLAN 20**.

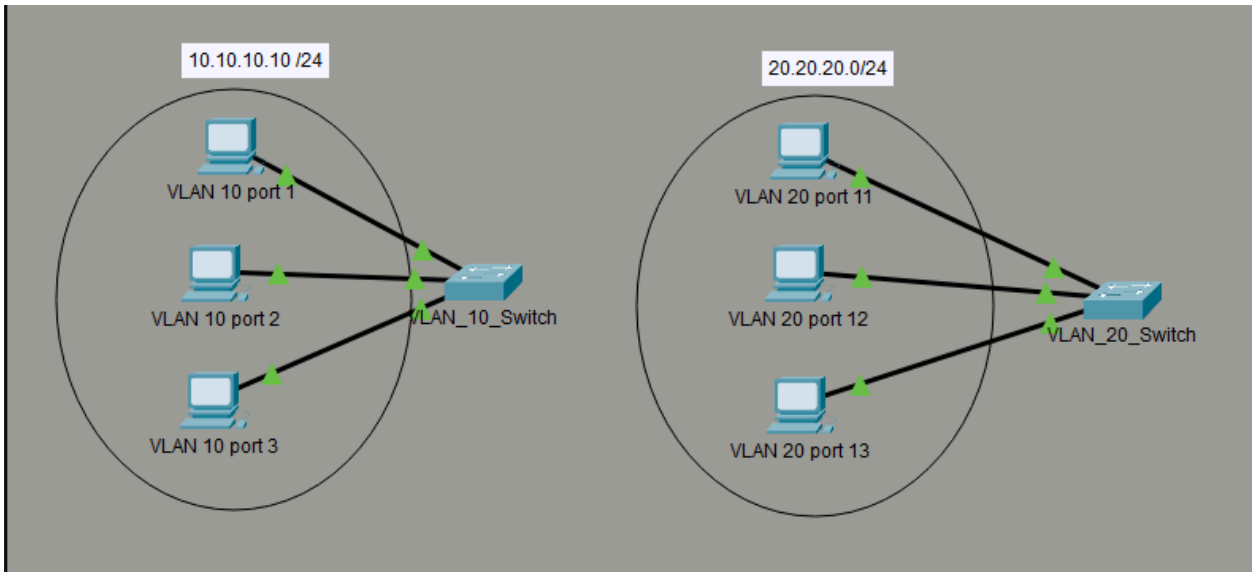
Physical Reality: Below is how the equipment actually looks on your desk or in a network rack. A single physical switch handling both groups of devices:



A few key things to notice here:

- The hosts in VLAN 10 cannot communicate with the hosts in VLAN 20.
- The hosts in each VLAN belong to entirely different IPv4 networks (`10.10.10.0/24` and `20.20.20.0/24`), which operate at **OSI Layer 3**.
- Since switches and VLANs operate strictly at **Layer 2** of the OSI model, they cannot move traffic between different subnets. Without a router (a Layer 3 device), these two groups are completely cut off from each other.

Logical View: Because of this Layer 2 isolation, the topology functionally operates as if you bought entirely separate hardware. Logically, it can be viewed like this:



By configuring VLANs, you have effectively chopped that single physical switch into **two completely separate, isolated logical switches**. Traffic from a host on `VLAN_10_Switch` cannot physically cross the gap to a host on `VLAN_20_Switch` without going up to a router first.

Verifying Access Ports inside the Switch

Now, let's look at how the switch actually views these assignments internally. By running the command **`show interface status`** on Switch0, we can see the exact layout of our logical boundaries:

```
Switch0#sh int status
Port      Name               Status       Vlan       Duplex  Speed Type
Fa0/1                        connected    10         a-full  a-100 10/100BaseTX
Fa0/2                        connected    10         a-full  a-100 10/100BaseTX
Fa0/3                        connected    10         a-full  a-100 10/100BaseTX
Fa0/4                        notconnect   10         auto    auto  10/100BaseTX
Fa0/5                        notconnect   10         auto    auto  10/100BaseTX
Fa0/6                        notconnect   10         auto    auto  10/100BaseTX
Fa0/7                        notconnect   10         auto    auto  10/100BaseTX
Fa0/8                        notconnect   10         auto    auto  10/100BaseTX
Fa0/9                        notconnect   10         auto    auto  10/100BaseTX
Fa0/10                       notconnect   10         auto    auto  10/100BaseTX
Fa0/11                       connected    20         a-full  a-100 10/100BaseTX
Fa0/12                       connected    20         a-full  a-100 10/100BaseTX
Fa0/13                       connected    20         a-full  a-100 10/100BaseTX
Fa0/14                       notconnect   20         auto    auto  10/100BaseTX
Fa0/15                       notconnect   20         auto    auto  10/100BaseTX
Fa0/16                       notconnect   20         auto    auto  10/100BaseTX
Fa0/17                       notconnect   20         auto    auto  10/100BaseTX
Fa0/18                       notconnect   20         auto    auto  10/100BaseTX
Fa0/19                       notconnect   20         auto    auto  10/100BaseTX
Fa0/20                       notconnect   20         auto    auto  10/100BaseTX
Fa0/21                       notconnect   1          auto    auto  10/100BaseTX
Fa0/22                       notconnect   1          auto    auto  10/100BaseTX
Fa0/23                       notconnect   1          auto    auto  10/100BaseTX
Fa0/24                       notconnect   1          auto    auto  10/100BaseTX
Gig0/1                       notconnect   1          auto    auto  10/100/1000BaseTX
Gig0/2                       notconnect   1          auto    auto  10/100/1000BaseTX
```

Key Takeaways from the CLI:

- **The Logical Boxes:** Notice that the range of **Fa0/1 - Fa0/10** is explicitly assigned to **VLAN 10**, and the range of **Fa0/11 - Fa0/20** is assigned to **VLAN 20**. The switch has effectively built two separate logical boxes inside its hardware.
- **Host Blindness:** The connected hosts still have absolutely no clue that they belong to a specific VLAN ID. Only the switch keeps track of this mapping.
- **Complete Isolation:** Because these ports belong to different logical boxes at Layer 2, hosts in different VLANs cannot pass traffic to each other. Without a Layer 3 device to route between them, they don't even know the other hosts exist.

How to Create, Assign, and Verify VLANs and Access Ports

Step 1: Create the VLANs

First, we must create the logical sandboxes in the switch's database

and give them a name.

```
Switch(config)#vlan 10
Switch(config-vlan)#name VLAN_10
Switch(config-vlan)#vlan 20
Switch(config-vlan)#name VLAN_20
Switch(config-vlan)#
```

Step 2: Assign the VLAN to a Single Switch Port

Next, we navigate to the physical interface and assign it to our logical box.

> **Important Concept:** There are **ONLY** two main operational switch port modes: **Access** and **Trunk**.

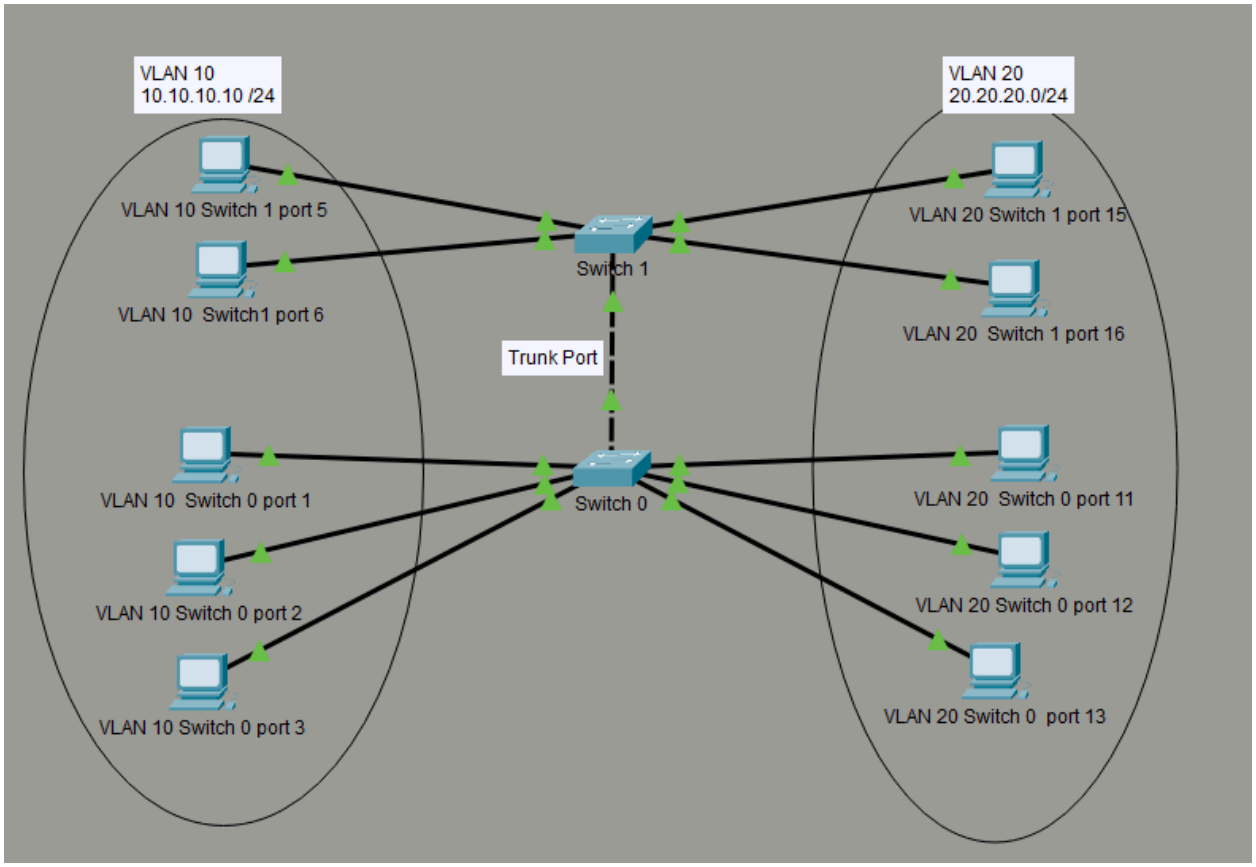
After designating a switch port as an Access Port, you need to explicitly tell it which specific VLAN to access. In this case, we point it to VLAN 10 for fa0/1, and VLAN 20 for fa0/11

```
Switch0(config)#int fa0/1
Switch0(config-if)#switchport mode access
Switch0(config-if)#switchport access vlan 10
Switch0(config-if)#int fa0/11
Switch0(config-if)#switchport mode access
Switch0(config-if)#switchport access vlan 20
```

Trunk Ports

So far, we have looked at VLANs within a single switch. What happens when we add more switches, each with more than one VLAN?

Look at the following network:



We need hosts on VLAN 10 Switch 0 to be able to communicate with hosts on VLAN 10 Switch 1, but not with hosts on VLAN 20 Switch 0 and Switch 1. And vice versa for all VLAN 20.

A **Trunk link** solves this by passing traffic from multiple VLANs at the same time, while keeping traffic from those VLANs completely separate from each other.

How Trunks Work: 802.1Q Tagging - IMPORTANT

Because a trunk port is a shared "multi-lane highway" carrying data from multiple networks over a single wire, the switches need a system to keep track of which frame belongs to which network. They achieve this using a protocol called **IEEE 802.1Q** (often called **Dot1Q**).

- **The Tag:** When a frame leaves Switch 0 to cross the trunk link, the switch pauses and injects a temporary **4-byte header (a tag)** directly into the middle of the standard Ethernet frame. Inside this tag is a 12-bit field containing the specific **VLAN ID** (e.g., VLAN 10).
- **The Hand-off:** The tagged frame travels across the trunk wire. When Switch 1 receives it, it reads the tag, sees that it belongs to VLAN 10, strips the tag away, and safely drops the clean, original frame *only* into the ports assigned to VLAN 10.
- **Host Blindness:** Remember that the end devices (the PCs) are completely blind to this. The tagging and untagging process happens exclusively while the data is actively moving on the wire between the two switches.

The Native VLAN

By default, a trunk link expects every single frame crossing it to have an 802.1Q tag. However, there is an exception to this rule called the **Native VLAN**.

- **The Rule:** If a switch receives an Ethernet frame on a trunk port that is completely **untagged** (it has no 802.1Q tag), it will not drop it. Instead, it automatically drops that frame into a default logical bucket called the Native VLAN. By default, this is **VLAN 1**.
- **The Configuration Trap:** For a trunk link to operate correctly, the Native VLAN **must match identically** on both ends of the connection. If Switch 0 thinks the Native VLAN is VLAN 1, but Switch 1 thinks the Native VLAN is VLAN 99, traffic will leak from one network into another, and the switches will flood your CLI with "Native VLAN Mismatch" warning errors.

How to Configure and Verify Trunk Links

To establish a trunk link between our switches, we navigate to the connecting interface (in this case, GigabitEthernet 0/1) and apply the configuration:

```
Switch0(config-if)#switchport mode trunk
Switch0(config-if)#switchport trunk native vlan 99
```

switchport mode trunk: Explicitly changes the operational state of the interface to a trunk link instead of an access port.

switchport trunk native vlan 99: Defines the designated Native VLAN for untagged frames crossing this wire. It is a standard industry best practice to always change the Native VLAN to a number other than the default VLAN 1 to prevent security vulnerabilities.

Verifying Your Trunk Status

To confirm that the trunk link is successfully active and negotiating correctly, run the **`show interface trunk`** command:Notice that this command shows the dot1q encapsulation, and the native VLAN for this trunk link. It also shows the VLANs currently being passed by this trunk link

```
Switch0#show interface trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      99

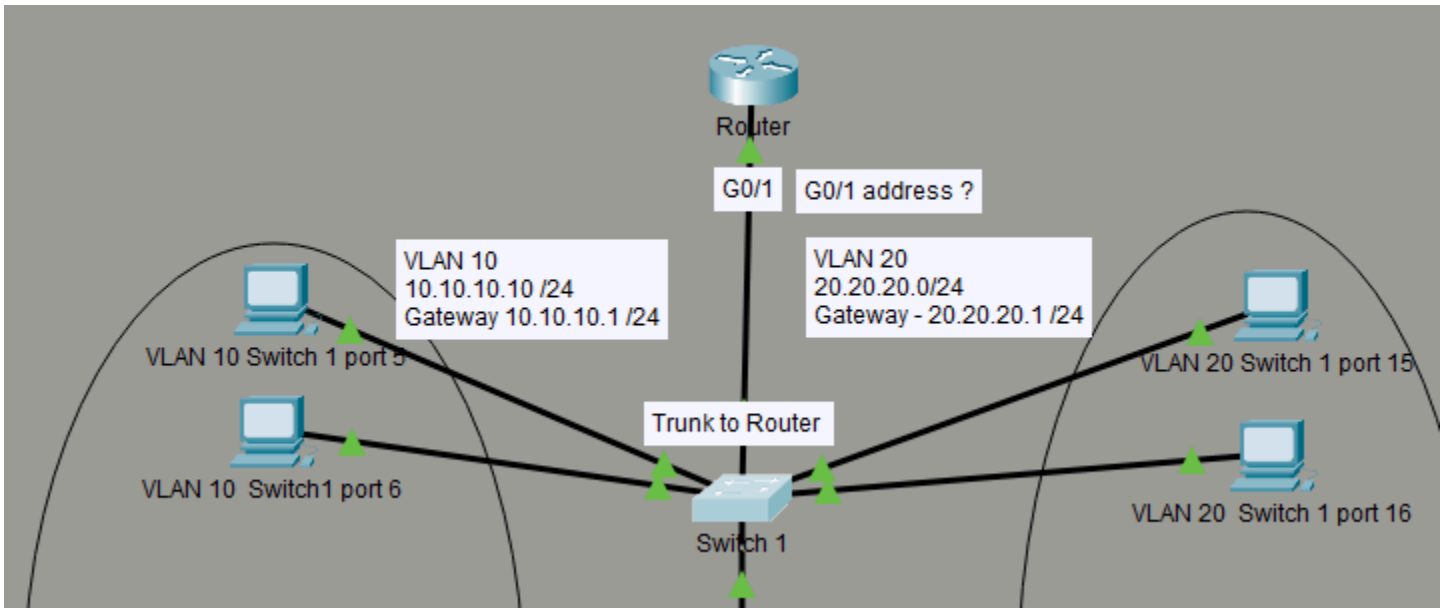
Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20
```

Subinterfaces and Router on a Stick

Now we need to address inter VLAN communication. If each VLAN behaves as if it is in its own private network, we need to enable layer 3 routing to move traffic between VLANs - in this case, we add a router



We know that each network needs its own Gateway Address for traffic to move into and out of it . In this case we have VLAN 10 and VLAN 20, and have a Gateway Address for each of them
But we only have one available Router interface, G0/1 .

We resolve this by creating sub interfaces on G0/1. We need a separate Gateway for each network

1 Verify that g0/1 is in a “no shut” state - it must be turned on for its sub-interfaces to work .

```
Router(config)#int g0/1
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, char
```

2 Create the subinterface - name it G0/1.10, the “ .10” matches the VLAN it is assigned to. This is not required but is good best practice

```
Router(config-if)#int g0/1.10
Router(config-subif)#
```

CLI Check: Note that your prompt changes to `Router(config-subif)#`, indicating you are now in **Configure - Sub-interface** CLI mode.

3 IMPORTANT - Before you can assign an IP gateway address, you **must** tell the sub-interface which 802.1Q tags it is responsible for reading

```
Router(config-subif)#encap dot1Q 10
```

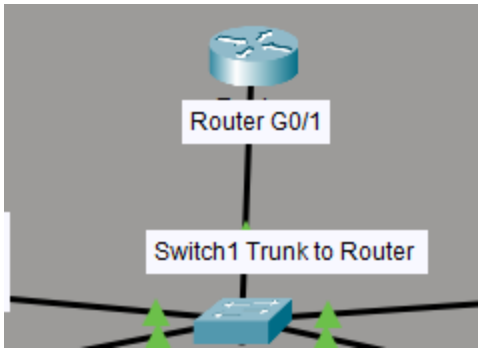
Note: This binds the sub-interface to **VLAN 10**, hence the "10" at the end of the command. If you skip this step and try to type an IP address first, the router will reject your command!

4 Set the IP address and description of the sub-interface

```
Router(config-subif)#ip address 10.10.10.1 255.255.255.0
Router(config-subif)#description Gateway to VLAN 10
```

`ip address 10.10.10.1 255.255.255.0`: This assigns the logical default gateway address that all host devices inside VLAN 10 will point to when they need to send packets outside their own subnet.
`description Gateway to VLAN 10`: A best-practice documentation command that labels the sub-interface, making it easily identifiable when running troubleshooting commands like `show ip interface brief` later on.

A Closer Look



A Closer Look at the Packet Lifecycle

Keep in mind what is actually happening behind the scenes as a packet moves through this topology:

- **Switch to Router:** When **Switch1** forwards traffic upstream to **Router interface G0/1**, it injects an 802.1Q VLAN tag into the frame header for all traffic (except frames belonging to the Native VLAN). This explicit tag is what allows the router to parse the frame and direct it to the exact logical sub-interface it belongs to.
- **Router to Switch:** Once the router processes the packet at Layer 3 and determines the destination network, it passes the data down to the next sub-interface. When the router transmits that traffic back out of its **G0/1 sub-interfaces**, it attaches a brand new 802.1Q VLAN tag. This ensures the trunk port on **Switch1** reads the tag, knows exactly which logical sandbox the traffic belongs to, and safely drops it into the correct destination ports.

Below is the configuration for G0/1.20 , along with a quick verification

```
Router(config)#int g0/1.20
Router(config-subif)#encap dot1q 20
Router(config-subif)#ip add 20.20.20.1 255.255.255.0
Router(config-subif)#desc Gateway to Vlan 20
Router(config-subif)#do sh ip int br
Interface              IP-Address      OK? Method Status
GigabitEthernet0/0     unassigned      YES unset  down
GigabitEthernet0/1     unassigned      YES unset  up
GigabitEthernet0/1.10  10.10.10.1      YES manual up
GigabitEthernet0/1.20  20.20.20.1      YES manual up
GigabitEthernet0/2     unassigned      YES unset  down
Vlan1                  unassigned      YES unset  administratively down
```